

Build: PDF Report Generator

Objective

Create a **PDF Report Generator** that uses saved:

- assessment outputs
- standard intelligence output
- AI narratives
- risk flags
- recommendations
- scenario results
- fragility profiles

to generate professional PDF reports for users.

Important rule:

PDF reports must not recalculate intelligence. They only format saved deterministic outputs and approved narratives.

Recommended Folder

`/src/lib/reports`

Suggested structure:

```
/src/lib/reports
├── types.ts
├── report-builder.ts
├── report-templates.ts
├── report-renderer.ts
├── report-storage.ts
├── report-generator.test.ts
└── index.ts
```

1. Report Types

```
export type ReportType =  
  | "business_health_report"  
  | "viability_report"  
  | "survival_report"  
  | "stability_report"  
  | "growth_report"  
  | "fragility_report"  
  | "scenario_report"  
  | "adaptive_decision_report"  
  | "investor_summary";
```

2. Input Contract

```
export interface PDFReportInput {  
  businessId: string;  
  reportType: ReportType;  
  
  intelligenceOutput: EnterpriseIntelligenceOutput;  
  
  narratives?: {  
    executiveNarrative?: string;  
    riskNarrative?: string;  
    recommendationNarrative?: string;  
    scenarioNarrative?: string;  
    confidenceNarrative?: string;  
  };  
  
  branding?: {  
    logoUrl?: string;  
    primaryColor?: string;  
    companyName?: string;  
  };  
};
```

```
metadata?: {  
  generatedBy?: string;  
  reportVersion?: string;  
};  
}
```

3. Output Contract

```
export interface PDFReportOutput {  
  reportId: string;  
  businessId: string;  
  reportType: ReportType;  
  title: string;  
  status: "generated" | "failed";  
  fileUrl?: string;  
  generatedAt: string;  
  sections: ReportSection[];  
}
```

```
export interface ReportSection {  
  sectionKey: string;  
  title: string;  
  content: string;  
  dataSource:  
    | "scores"  
    | "risks"  
    | "recommendations"  
    | "fragility"  
    | "scenario"  
    | "narrative";  
}
```

4. Report Sections

Business Health Report

1. Cover Page
2. Executive Summary
3. Business Intelligence Score
4. Engine Score Breakdown
5. Fragility Index
6. Top Risks
7. Recommendations
8. Scenario Insights, if available
9. Confidence & Data Quality Note
10. Next Best Actions

Scenario Report

1. Scenario Summary
2. Base vs Simulated Scores
3. Risk Changes
4. Recommendation Changes
5. Decision Path
6. Strategic Interpretation

Fragility Report

1. Fragility Overview
2. Fragility Dimensions
3. Top Fragility Factors
4. Risk Propagation Paths
5. Mitigation Recommendations

5. Renderer Options

Use one of these:

Recommended for Web App

React PDF / @react-pdf/renderer

Good for:

- Next.js
- React components
- branded layouts

Alternative

HTML → PDF using Puppeteer

Good for:

- richer design
- CSS control
- report previews

For MVP, use:

HTML template + Puppeteer

6. Database Tables

```
CREATE TABLE reports (  
  id UUID PRIMARY KEY,  
  business_id UUID NOT NULL,  
  report_type VARCHAR NOT NULL,  
  title VARCHAR NOT NULL,  
  status VARCHAR DEFAULT 'generating',
```

```
source_assessment_id UUID,  
source_simulation_run_id UUID,  
file_url TEXT,  
generated_by UUID,  
created_at TIMESTAMP DEFAULT NOW(),  
completed_at TIMESTAMP  
);  
CREATE TABLE report_sections (  
id UUID PRIMARY KEY,  
report_id UUID REFERENCES reports(id),  
section_key VARCHAR NOT NULL,  
section_title VARCHAR NOT NULL,  
content_text TEXT,  
data_source VARCHAR,  
section_order INT,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

7. API Endpoint

POST /businesses/:id/reports/generate

Body:

```
{  
  "reportType": "business_health_report",  
  "sourceAssessmentId": "uuid",  
  "includeNarratives": true,  
  "branding": {  
    "companyName": "EnterprateAI"  
  }  
}
```

Response:

```
{  
  "success": true,  
  "reportId": "uuid",
```

```
"status": "generated",  
"fileUrl": "https://..."  
}
```

Retrieve:

GET /businesses/:id/reports/:reportId

8. Report Builder Logic

```
Load saved assessment output  
↓  
Load saved narratives  
↓  
Select report template  
↓  
Build report sections  
↓  
Render HTML  
↓  
Convert to PDF  
↓  
Upload to storage  
↓  
Save report metadata  
↓  
Return file URL
```

9. Report Design Requirements

PDF must include:

- professional cover page
- business name
- report type

- generated date
- executive summary
- score cards
- risk table
- recommendation table
- confidence note
- disclaimer: “Decision-support report, not financial advice”

10. Codex Prompt

Build the EnterpriseAI PDF Report Generator in `/src/lib/reports`.

The generator must use saved `EnterpriseIntelligenceOutput` and AI narratives.

It must not recalculate scores or call any assessment engine.

Create:

- `types.ts`
- `report-builder.ts`
- `report-templates.ts`
- `report-renderer.ts`
- `report-storage.ts`
- `report-generator.test.ts`
- `index.ts`

Support report types:

- `business_health_report`
- `viability_report`
- `survival_report`
- `stability_report`
- `growth_report`
- `fragility_report`
- `scenario_report`
- `adaptive_decision_report`
- `investor_summary`

Use HTML template + Puppeteer for MVP PDF generation.

Implement:

- PDFReportInput
- PDFReportOutput
- section builder
- HTML renderer
- PDF renderer
- storage upload placeholder
- database persistence for reports and report_sections
- API endpoint POST /businesses/:id/reports/generate
- API endpoint GET /businesses/:id/reports/:reportId

Add unit tests for:

- business health report
- scenario report
- fragility report
- missing narrative fallback
- failed PDF generation

Important:

- Do not recalculate scores.
- Do not call LLM directly.
- Use only saved deterministic output and approved narratives.
- Include confidence and data quality note.
- Include disclaimer.